

Setting up Hope Beacon with an AI assistant

For IT people who would rather describe the job than type it.

Everything in [START-HERE.md](#) can be handed to an AI assistant. This document is about doing that *well*, because the ways it goes wrong are specific and predictable.

Read [START-HERE.md](#) first — this assumes you know what is being set up and why.

You can hand most of this document to an AI assistant and have it do the typing. This section is about doing that *well*, because the failure modes are specific.

What to use

Any of Claude, ChatGPT, Gemini, GitHub Copilot, Cursor. The free tiers are enough for setup. There is a longer list in [AI-TOOLKIT.md](#).

The one thing that matters most

Give it the repository, not a description of the repository.

An AI that has read `supabase/migrations/0001_core_schema.sql` knows your exact security rules. One that has only been told "it's a church app with Supabase" will invent a plausible schema that does not match yours, and everything it writes afterwards will be subtly wrong in a way that is hard to see.

Tools like Claude Code, Cursor and Copilot Workspace read the actual files. Use one of those if you can. If you are using a chat window, paste the real file.

Prompts that work

Setting up:

I am setting up Open Hope Beacon from `github.com/klydo131/open-hope-beacon` . Read `docs/START-HERE.md` and `docs/SETUP.md` . I am on Part 3 Step 3 and the SQL editor returned this error: `<paste the exact error>` . What went wrong and what do I run instead?

Making a change:

In this repository, I want Explorers to be able to mark a lesson as finished. Before writing anything: read `supabase/migrations/0012_lessons_and_notifications.sql` and `lib/live/data.ts` , and tell me which files you would change and why. Do not write code yet.

That last sentence is the important one. Ask for the plan first. It takes ten seconds to read a plan and a long time to unpick a confident wrong change.

Understanding something:

Explain what `supabase/migrations/0003_invite_approval_gate.sql` does, in plain English, as if I do not know SQL. What could go wrong if I removed it?

The rules, and they are not optional

Never paste a key into a chat window. Not the `service_role` key, not the Brevo key, not your database password. Say `<my API key>` instead. If you have already pasted one, go and rotate it now — it is a button in the same place you created it.

Make it show you, not tell you. "Did that work?" gets you a cheerful yes. "Run the query that proves it and show me the output" gets you the truth. This document exists partly because an AI reported a fixed invitation system twice before anyone clicked a link.

Ask for the negative test. If it says a security rule works, ask it to prove the rule *blocks* what it should block. A test where nothing fails has proved nothing. Every security fix in this project is checked that way, and the reason is that a test suite once passed while refusing four things for a completely unrelated reason.

It will be confidently wrong sometimes. Not occasionally — regularly, and most convincingly on the things it half-knows. Check anything that touches money, security, or somebody's private conversation.

A working session, start to finish

Copy this into an assistant that can read files (Claude Code, Cursor, Copilot Workspace), one message at a time. Wait for each answer.

1. Orient it

Clone `https://github.com/klydo131/open-hope-beacon` and read what the four services are for, and what order I have to do things in. Do not change anything yet.

If the summary is wrong, stop. Everything after this depends on it.

2. Get it running locally

Get the app running on my machine with no database, the way the README describes. Tell me the exact commands and what I should see.

3. The database

I have created a Supabase project. Walk me through applying every migration in `supabase/migrations/` in order. For each file, tell me in one line what it adds, so I know what I am running. Warn me about anything irreversible.

4. Prove the security rules — do not skip this

Write me SQL that proves a Guide cannot read another Guide's Explorer. Include a positive control: a query that SHOULD succeed, so that if the permission check is broken in the other direction I will see it. Show me the output of both.

A test where nothing fails has proved nothing. This is the single most valuable thing an assistant can do for you here.

5. Email

Set up the invitation email through Brevo, following `docs/EMAIL.md`. I will paste the secrets into Supabase myself — give me the names and tell me where they go. Do not ask me for the key.

6. Deploy

Walk me through deploying to Vercel and tell me exactly which environment variables to set. Then tell me how to verify the deployment actually works, as a test I run rather than a thing you assert.

Things to watch for

"It's working now." Ask how it knows. This project's own history has a fixed invitation system reported as fixed twice, while every link it produced still failed on arrival, because nobody clicked one.

Confident SQL against a schema it has not read. If it did not open

Security rules edited to make an error go away. "Permission denied" usually means the rules work. Weakening them to clear the error is how a church's private conversations end up readable by anyone holding a key that ships in every visitor's browser.

Any request for a secret. No assistant needs your `service_role` key, your Brevo key, or your database password. If one asks, say no and paste

If you do not have an AI assistant

Everything here can be done by hand. Part 7 of [START-HERE.md](#) is the manual path with every command, and Part 8 is the same journey for someone who has never opened a terminal.